

Revised Report — What Happens Through TrainingService

Overview

TrainingService is the offline industrial ML pipeline of the MDF1 ML platform.

Its responsibility is to transform raw MDF1 production/lab exports into:

- validated industrial datasets,
- engineered process features,
- trained ML models,
- explainability reports,
- and production-ready artifacts for PredictionService.

The pipeline was progressively refactored toward:

- centralized configuration,
- modular architecture,
- reduced redundancy,
- industrial traceability,
- production-aware evaluation,
- and explainable ML behavior.

High-Level Pipeline Flow

```
Raw MDF1 Excel Files
↓
Ingestion & Schema Parsing
↓
Validation
↓
Cleaning & Normalization
↓
Feature Engineering
↓
Audit / EDA
↓
Process Explainability Reports
↓
Time-Based Cross Validation
↓
Model Training & Hyperparameter Tuning
↓
Ensemble Evaluation
```

↓
Best Model Selection
↓
Artifact Publishing
↓
Final Explainability & Recommendation Reports

1. Raw MDF1 Data Ingestion

TrainingService loads all `.xlsx` files from:

`data/raw/`

The architecture evolved from:

- single hardcoded Excel file to:
- dynamic multi-file ingestion using wildcard patterns.

This allows:

- very large exports,
- split exports,
- future scalable ingestion.

The ingestion layer:

- reads KSoft exports,
 - parses multi-row headers,
 - extracts human-readable names,
 - extracts technical FQTS names,
 - builds the modeling dataframe,
 - generates column mappings,
 - saves flattened parquet/csv backups.
-

2. Validation Layer

Before modeling, the pipeline validates:

- required targets,
- feature existence,
- duplicate columns,
- invalid schemas,
- missing time columns,

- dataframe integrity.

This is important because industrial exports may contain:

- inconsistent schemas,
- operator-generated anomalies,
- mixed datatypes,
- missing process signals,
- incomplete lab data.

The validator acts as a protective industrial gate before ML training begins.

3. Cleaning & Data Normalization

The cleaning layer:

- normalizes text-based null values,
- parses timestamps,
- coerces numeric columns safely,
- removes invalid target rows,
- deduplicates production records,
- removes highly missing features.

The pipeline uses:

```
errors="coerce"
```

for industrial-safe conversion behavior.

This means:

- invalid values do not crash the pipeline,
- problematic values become `NaN`,
- processing continues safely.

This is especially important for MDF1/KSoft exports containing:

- `--`
- `NULL`
- broken timestamps
- mixed object values.

The parquet export layer was additionally hardened to safely serialize mixed industrial object columns.

4. Feature Engineering

The feature engineering layer transforms raw MDF process signals into higher-level process intelligence features.

Examples include:

Pressure Aggregates

```
pressPressureGlobal_mean  
pressPressureGlobal_std  
pressPressureFront_mean  
pressPressureMid_mean  
pressPressureEnd_mean  
pressPressureFrontEnd_diff
```

Temperature Aggregates

```
tempGlobal_mean  
tempGlobal_range
```

Thickness Aggregates

```
thicknessClosed_mean  
thicknessClosed_std  
thicknessClosed_range
```

These features better represent MDF process behavior than raw sensor channels alone.

5. Audit & Industrial EDA

The audit layer generates:

- row/column statistics,
- time ranges,
- target summaries,
- top process correlations.

Correlation analysis currently uses:

```
Pearson correlation
```

through pandas correlation methods.

Purpose:

- fast statistical screening,
- industrial process understanding,
- baseline feature relevance analysis.

However, because MDF processes are nonlinear and interaction-heavy, the architecture evolved beyond pure correlation into:

- ML feature importance,
 - SHAP explainability,
 - and combined explainability scoring.
-

6. Production-Aware Time-Based Cross Validation

A major architectural improvement was the transition from:

- naive random splitting to:
- production-aware time-based validation.

Why?

Because MDF1 production data is NOT IID (Independent and Identically Distributed).

Industrial process data evolves over time:

- raw material changes,
- recipe changes,
- moisture variation,
- operator changes,
- machine behavior drift.

Therefore the pipeline now:

- trains on older production,
- validates on newer production,
- evaluates future generalization behavior.

The service implements:

- chronological train/validation/test splitting,
- expanding-window time-series CV.

This better simulates real industrial deployment conditions and reduces leakage risk.

7. Model Training

For each primary MDF1 lab target:

```
labDensityAverage  
labBendingAvg  
labEModulAvg  
labTensileAvg  
labSurfaceSoundnessAvg
```

the pipeline trains multiple algorithms:

```
Random Forest  
XGBoost  
LightGBM
```

The architecture supports:

- modular algorithm configuration,
- centralized settings,
- reusable training orchestration,
- artifact versioning.

8. Feature Selection

Feature selection is performed ONLY using training data to prevent leakage.

The service:

- selects top correlated features,
- merges domain-forced industrial features,
- removes unstable/invalid signals.

The architecture intentionally combines:

- statistical relevance,
 - domain knowledge,
 - engineered process features.
-

9. XGBoost Hyperparameter Tuning

XGBoost tuning includes:

- learning_rate,
- max_depth,
- gamma,
- subsampling,
- regularization,
- early stopping.

Regularization

The pipeline tunes:

```
reg_alpha  
reg_lambda
```

Purpose:

- reduce overfitting,
- stabilize learning,
- reduce noise memorization,
- improve future production generalization.

Early Stopping

The service uses validation-aware early stopping to stop training when:

- validation performance stops improving.

This prevents:

- over-training,
- noise memorization,
- unstable industrial models.

10. Ensemble Evaluation

The pipeline additionally evaluates:

- XGBoost + Random Forest weighted ensembles.

The ensemble is accepted only if:

- validation performance meaningfully improves over single models.

This introduces:

- industrial robustness,
 - diversified prediction behavior,
 - controlled ensemble governance.
-

11. Explainability Architecture

The explainability layer combines three levels:

A. Correlation

Statistical relationship between process variables and targets.

B. ML Feature Importance

Using:

- Random Forest importance,
- XGBoost importance,
- LightGBM importance.

These measure how strongly trained models rely on features.

C. SHAP Explainability

SHAP computes:

- per-row feature contribution,
- average absolute feature influence,
- global feature importance ranking.

SHAP ranking is based on:

```
mean_abs_shap = np.abs(shap_values).mean(axis=0)
```

Meaning:

- average prediction influence magnitude.

The architecture intentionally combines:

- correlation,
- ML importance,
- SHAP influence

into a final explainability layer.

This provides:

- statistical insight,
 - model insight,
 - prediction-behavior insight.
-

12. Artifact Publishing

Each trained model produces a production-ready artifact bundle:

```
model.joblib  
preprocessor.joblib  
features.json  
metrics.json  
metadata.json  
feature_importance.csv  
feature_importance.xlsx
```

Artifacts are stored under:

```
artifacts/<model_name>/
```

These are later consumed by PredictionService.

13. Best Model Registry

The pipeline generates:

```
best_models_registry.json
```

This registry defines:

- which model serves each target,
- whether serving uses:
 - single model,
 - weighted ensemble,
- artifact locations,
- evaluation metrics.

This acts as the bridge between:

- TrainingService and

- PredictionService.
-

14. Logging & Pipeline Governance

The architecture now includes:

- centralized logging,
- decorators,
- timing monitoring,
- structured pipeline execution tracking.

Pipeline decorators automatically log:

- STARTED,
- FINISHED,
- FAILED,
- execution duration.

Logs are written into:

```
logs/training.log
```

This significantly improves:

- traceability,
 - debugging,
 - industrial maintainability.
-

15. Architectural Refactoring Improvements

The service was progressively refactored to:

- reduce redundancy,
- centralize settings,
- separate responsibilities,
- standardize utilities,
- modularize reporting,
- improve maintainability.

Examples:

- settings centralized in `settings.py`
- reusable filesystem utilities
- reusable logging utilities
- decorator-based orchestration
- reusable report writers

- centralized algorithm configuration

This moved the project from:

- notebook-style ML experimentation

toward:

- industrial-grade ML platform architecture.
-

Current Architectural Position

TrainingService now functions as:

industrial offline ML orchestration platform

with:

- MDF1-specific process intelligence,
- production-aware validation,
- explainability layers,
- artifact governance,
- modular industrial architecture,
- and PredictionService integration readiness.

It is significantly more advanced than a basic ML training script and already resembles an enterprise industrial AI pipeline architecture.

16. Module, Class, and Functionality Overview

This section explains the main TrainingService modules and their responsibilities, including the specific methods or algorithms applied inside each layer.

16.1 `config/settings.py`

Functionality

This file acts as the central configuration layer of TrainingService.

It contains:

- project paths,
- data paths,
- artifact paths,

- report paths,
- random seed,
- split ratios,
- model hyperparameter defaults,
- XGBoost tuning ranges,
- explainability settings,
- report configuration.

Why It Matters

This avoids duplicated constants across the service.

Instead of hardcoding values inside classes or functions, the service imports values from `settings.py`.

Example:

```
RANDOM_STATE = 42
TEST_SIZE = 0.2
VALID_SIZE = 0.2
SHAP_MAX_ROWS = 500
```

This makes the TrainingService easier to maintain and safer for production-style execution.

16.2 `config/columns.py`

Functionality

This file defines MDF1-specific schema knowledge.

It contains:

- target columns,
- model feature columns,
- ID columns,
- time column,
- pressure columns,
- temperature columns,
- thickness columns,
- process feature groups.

Why It Matters

This keeps MDF1 process knowledge centralized.

For example, process groups are later used in explainability reports to map features into industrial groups such as:

- press process,
- fiber preparation,
- glue/chemical process,
- temperature process,
- mat forming.

16.3 data/ingest_raw_ksoft.py

Functionality

This module loads raw KSoft Excel exports from:

```
data/raw/
```

It supports multi-file ingestion using:

```
RAW_FILE_PATTERN = "*.xlsx"
```

Main responsibilities:

- find all Excel files,
- load raw sheets,
- preserve KSoft multi-row header structure,
- extract human-readable names,
- extract technical FQTTS names,
- make duplicate columns unique,
- build the modeling dataframe,
- save flattened data,
- save column mapping.

Specific Method Used

The ingestion logic uses pandas Excel loading:

```
pd.read_excel(..., header=None, engine="openpyxl")
```

This is necessary because the KSoft export uses multi-row headers, where:

```
row 1 = human-readable label  
row 2 = technical FQTTs column name  
row 3 onward = production/lab data
```

16.4 data/validator.py

Functionality

This module validates the dataframe before modeling.

It checks:

- required columns,
- target availability,
- missing columns,
- duplicate columns,
- invalid datatypes,
- time column quality,
- missing value ratios.

Specific Method Used

It uses safe conversion methods such as:

```
pd.to_numeric(..., errors="coerce")  
pd.to_datetime(..., errors="coerce")
```

This protects the pipeline from crashing when industrial data contains invalid values.

16.5 data/cleaner.py

Functionality

This module cleans raw MDF1 data before feature engineering and training.

Main responsibilities:

- normalize text-based null values,
- parse production time,
- convert model features and targets to numeric,
- remove rows without primary target values,
- deduplicate records,
- sort data chronologically,
- drop features with excessive missing values,

- save cleaned data.

Specific Method Used

The cleaning process uses `errors="coerce"` during numeric and datetime conversion.

This means invalid values are converted to `NaN` or `NaT` instead of raising errors.

The parquet saving logic also converts mixed object columns to strings before saving. This prevents PyArrow serialization errors caused by mixed datatypes in KSoft columns.

16.6 `data/splitter.py`

Functionality

This module provides two splitting strategies:

Random Split

```
split_train_val_test()
```

Uses random shuffling to create:

- train set,
- validation set,
- test set.

This is useful for baseline benchmarking.

Time-Based Split

```
split_time_based()
```

Sorts data by production time and creates chronological splits:

```
older data → train  
middle data → validation  
newer data → test
```

Specific Method Used

The time-based method is production-aware because it simulates real deployment:

```
train on historical production  
predict future production
```

This reduces leakage risk compared with random splitting.

16.7 features/engineering.py

Functionality

This module creates MDF1-specific engineered features.

It transforms raw sensor/process columns into higher-level process indicators.

Examples:

```
pressPressureGlobal_mean  
pressPressureGlobal_std  
pressPressureFront_mean  
pressPressureMid_mean  
pressPressureEnd_mean  
pressPressureFrontEnd_diff  
tempGlobal_mean  
thicknessClosed_range
```

Specific Methods Used

It uses row-wise statistical aggregation:

- mean,
- standard deviation,
- minimum,
- maximum,
- range,
- zone-based grouping.

These engineered features represent process behavior more clearly than isolated raw sensor values.

16.8 features/selectors.py

Functionality

This module selects usable and relevant features for training.

It removes unusable features such as:

- missing-only columns,
- constant columns,
- target leakage columns,
- lab-result columns,
- unstable proxy columns.

It also appends forced production features defined by domain knowledge.

Specific Method Used

The main selection method uses Pearson correlation:

```
df.corr(numeric_only=True)
```

Features are ranked by absolute correlation with the target.

Important point:

```
feature selection is performed using train data only
```

This prevents leakage from validation/test data.

16.9 eda/audit.py

Functionality

This module generates an initial data audit report.

It summarizes:

- row count,
- column count,
- time range,
- target statistics,
- top correlations with each target.

Specific Method Used

It uses Pearson correlation for fast statistical screening.

This is useful for early industrial understanding, but it does not replace ML importance or SHAP explainability.

16.10 `models/registry.py`

Functionality

This module has two main responsibilities:

1. create model pipelines,
2. save the best model registry.

Model Factory

```
get_regression_model()
```

Creates sklearn-style pipelines such as:

```
SimpleImputer + RandomForestRegressor  
SimpleImputer + XGBRegressor  
SimpleImputer + LGBMRegressor
```

Best Model Registry

```
save_best_models_registry()
```

Creates:

```
artifacts/best_models_registry.json
```

This file tells PredictionService which artifact should be served for each target.

Specific Method Used

The model factory uses a median imputation pipeline:

```
SimpleImputer(strategy="median")
```

This helps handle missing industrial process values safely.

16.11 `models/model_catalog.py`

Functionality

This module defines which algorithms will be trained.

It returns a list of algorithm configurations such as:

- Random Forest,
- XGBoost,
- LightGBM.

Each configuration includes:

- algorithm name,
- artifact suffix,
- model parameters,
- notes.

Why It Matters

This avoids scattering model configuration inside the training loop.

The training pipeline can iterate over configured algorithms consistently.

16.12 `models/metrics.py`

Functionality

This module calculates regression metrics.

It returns:

```
MAE
RMSE
R2
```

Specific Methods Used

It uses sklearn metrics:

```
mean_absolute_error
mean_squared_error
r2_score
```

Meaning

- MAE: average absolute prediction error,
 - RMSE: penalizes larger errors more strongly,
 - R²: explains how much target variance is captured by the model.
-

16.13 `models/xgb_tuner.py`

Functionality

This module tunes XGBoost hyperparameters.

It searches different combinations of:

- `learning_rate`,
- `max_depth`,
- `gamma`,
- `subsample`,
- `colsample_bytree`,
- `reg_alpha`,
- `reg_lambda`,
- `min_child_weight`,
- `n_estimators`.

Specific Algorithm Used

It uses random hyperparameter sampling:

`ParameterSampler`

Instead of testing every possible combination, it samples a limited number of configurations.

Early Stopping

XGBoost also uses early stopping:

stop training when validation score does not improve

This prevents overfitting and excessive tree growth.

16.14 `models/time_cv.py`

Functionality

This module performs production-aware time-series cross-validation.

It creates expanding-window folds:

```
Fold 1: older data → next period  
Fold 2: more older data → later period  
Fold 3: even more older data → newest period
```

Specific Method Used

The method is expanding-window validation.

This is suitable for industrial production because the model is always evaluated on data that happens after the training period.

This reduces future leakage.

16.15 `models/cv_runner.py`

Functionality

This module runs cross-validation for a specific model type.

It supports:

- RF folds,
- XGB folds,
- XGB/RF ensemble folds.

For each fold it:

- selects features from training data,
- fits the model,
- predicts validation data,
- calculates fold metrics.

Specific Methods Used

- Random Forest regression,
- XGBoost regression,
- weighted ensemble evaluation,
- time-series CV from `time_cv.py`.

16.16 `models/cv_orchestrator.py`

Functionality

This module coordinates CV execution for one target across multiple model kinds.

It calls:

```
run_model_cv_summary()
```

for each configured model type.

Why It Matters

It keeps the main pipeline cleaner and separates CV orchestration from model training logic.

16.17 `models/ensemble.py`

Functionality

This module evaluates weighted ensemble predictions.

Formula:

```
prediction = weight * model_a + (1 - weight) * model_b
```

In this project:

```
model_a = XGBoost  
model_b = Random Forest
```

Specific Method Used

It tests multiple candidate weights and selects the best validation score.

A penalty is applied to avoid overly extreme weights.

This improves stability and prevents over-trusting one model.

16.18 `models/ensemble_selection.py`

Functionality

This module decides whether the XGB/RF ensemble should be accepted.

It compares:

```
ensemble validation R2  
vs  
best single model validation R2
```

The ensemble is accepted only if it improves by at least a configured minimum gain.

Why It Matters

This prevents creating ensemble artifacts that do not meaningfully improve performance.

16.19 `models/train.py`

Functionality

This module contains model-support functions such as:

- preparing modeling datasets,
- evaluating multi-target regression,
- extracting feature importance.

Specific Method Used

Feature importance extraction reads:

```
feature_importances_
```

from tree-based models such as:

- Random Forest,
 - XGBoost,
 - LightGBM.
-

16.20 pipeline/training_pipeline.py

Functionality

This is the core single-target training pipeline.

For one target and one algorithm, it:

- validates required columns,
- performs train/validation/test split,
- selects train-only features,
- trains the model,
- tunes XGBoost when needed,
- calculates validation/test metrics,
- extracts feature importance,
- saves model artifacts,
- returns a `TrainingPipelineResult`.

Important Class

`TrainingPipelineResult`

This class stores:

- algorithm name,
- target name,
- artifact name,
- metrics,
- artifact directory,
- validation predictions,
- test predictions.

This object is also used later for ensemble evaluation and registry selection.

16.21 models/artifact_saver.py

Functionality

This module saves model artifacts.

Each artifact bundle contains:

```
model.joblib  
features.json  
metrics.json  
metadata.json
```



```
feature_importance.csv  
feature_importance.xlsx
```

Why It Matters

This is the contract between TrainingService and PredictionService.

PredictionService later loads:

- saved model,
- saved feature order,
- saved metadata,
- best model registry.

16.22 `reports/process_impact_report.py`

Functionality

This module builds process impact reports.

It maps process features to process groups and calculates their relationship with each target.

Specific Method Used

It uses Pearson correlation between each process feature and target.

It also calculates:

- missing ratio,
- valid pair count,
- impact direction,
- absolute correlation,
- process group.

16.23 `reports/ml_process_importance_report.py`

Functionality

This module combines:

- process impact reports,
- ML feature importance files.

It answers:

Which process-group features are important according to trained ML models?

Specific Method Used

It joins process correlation data with model feature importance using:

```
target + feature
```

16.24 reports/shap_explainability_report.py

Functionality

This module calculates SHAP explainability from saved model artifacts.

It loads:

```
model.joblib  
features.json  
metadata.json
```

Then computes SHAP values for available feature columns.

Specific Algorithm Used

It uses:

```
shap.TreeExplainer
```

This is suitable for tree-based models such as:

- Random Forest,
- XGBoost,
- LightGBM.

SHAP Ranking

SHAP rank is calculated by:

```
mean_abs_shap = np.abs(shap_values).mean(axis=0)
```

Then features are sorted from highest to lowest influence.

Rank 1 means the strongest average prediction influence.

16.25 `reports/final_explainability_report.py`

Functionality

This module merges:

- process correlation,
- ML importance,
- SHAP importance.

It creates the final explainability score:

```
final_explainability_score = absolute_correlation + ml_importance +  
mean_abs_shap
```

Why It Matters

This combines three viewpoints:

- statistical relationship,
 - model usage,
 - prediction contribution.
-

16.26 `reports/process_group_summary.py`

Functionality

This module summarizes explainability at process-group level.

It helps identify which industrial process groups have the strongest effect per target.

Examples:

- press process,
- fiber preparation,
- glue/chemical process,
- temperature process.

Specific Method Used

It aggregates:

- average ML importance,

- average absolute correlation,
 - top feature per group,
 - missing ratio,
 - valid pair counts.
-

16.27 `reports/process_recommendation_report.py`

Functionality

This module converts final explainability into human-readable process recommendations.

It creates comments such as:

- process interpretation,
- feature comment,
- data quality comment,
- recommended action.

Important Note

These are decision-support recommendations only.

They are not automatic control instructions.

16.28 `utils/logging_utils.py`

Functionality

This module creates centralized loggers.

It writes logs to:

```
logs/training.log
```

Why It Matters

It provides:

- traceability,
 - debugging support,
 - error tracking,
 - training history.
-

16.29 `utils/decorators.py`

Functionality

This module provides AOP-style decorators.

The main decorator is:

```
@pipeline_step(...)
```

It automatically logs:

- start,
- end,
- failure,
- elapsed time.

Why It Matters

This avoids repeated timing/logging code across classes and functions.

16.30 `utils/file_utils.py`

Functionality

This module contains reusable file operations.

Examples:

- read JSON,
- write JSON,
- write CSV,
- write Excel,
- create directories,
- safe file naming,
- write paired CSV/XLSX reports.

Why It Matters

It removes repeated file-saving logic from report modules.

16.31 `utils/artifact_cleaner.py`

Functionality

This module optionally cleans old artifacts before a new training run.

Controlled by settings:

ENABLE_ARTIFACT_CLEAN_BEFORE_RUN

If enabled, previous artifacts are deleted before new artifacts are generated.

This ensures clean training output when required.

16.32 `utils/seed.py`

Functionality

This module sets random seeds for reproducibility.

It sets:

- Python random seed,
- NumPy random seed,
- `PYTHONHASHSEED`.

Why It Matters

It helps keep repeated training runs more stable and reproducible.

16.33 `main.py`

Functionality

`main.py` is the orchestration entry point.

It should not contain business logic.

It coordinates:

- seeding,
- optional artifact cleanup,
- ingestion,
- validation,

- cleaning,
- feature engineering,
- audit,
- process reports,
- training loop,
- ensemble checks,
- comparison reports,
- best model registry,
- SHAP reports,
- final explainability,
- process recommendations,
- timing reports.

Architectural Role

The role of `main.py` is orchestration only.

Detailed logic belongs inside dedicated modules.

17. Summary of Main Algorithms and Methods Used

Data Preparation

- pandas Excel ingestion,
- schema validation,
- safe numeric coercion,
- safe datetime coercion,
- missing-value handling,
- median imputation.

Feature Engineering

- row-wise aggregation,
- pressure zone grouping,
- temperature aggregation,
- thickness aggregation,
- range and difference features.

Feature Selection

- Pearson correlation,
- absolute correlation ranking,
- train-only feature selection,
- domain-forced feature inclusion.

Model Algorithms

- Random Forest Regression,
- XGBoost Regression,
- LightGBM Regression,
- weighted XGB/RF ensemble.

Model Evaluation

- MAE,
- RMSE,
- R^2 ,
- validation/test split,
- expanding-window time-series CV.

Explainability

- Pearson correlation,
- tree-based feature importance,
- SHAP TreeExplainer,
- mean absolute SHAP ranking,
- final explainability score.

Governance

- artifact bundles,
- best model registry,
- metadata tracking,
- logging,
- decorators,
- timing reports.